

proyecto invernadero artificial

Desarrollo de Software Concurrente Basado en Programación Orientada a Objetos, Arquitectura Desacoplada y Sincronización con APIs Climatológicas en Tiempo Real

oscar david obando , Jonathan Ramirez Velez ,Juan Sebastian Ladino, Esteban Henao

programacion 2

Universidad Tecnológica de Pereira

Pereira, Risaralda, Colombia

Año Académico 2026

Resumen—El presente documento detalla la arquitectura, diseño e implementación de un sistema avanzado de simulación y monitoreo para invernaderos inteligentes bajo el contexto de la Agricultura 4.0. Empleando el paradigma de Programación Orientada a Objetos (POO), procesamiento multihilo masivo y consumo de APIs meteorológicas externas, la plataforma se consolida como un gemelo digital capaz de predecir riesgos biológicos y optimizar el microclima para cultivos simultáneos. Este informe técnico profundiza en las estructuras de datos cíclicas, los modelos trigonométricos de la interfaz y las ecuaciones predictivas de riesgo fitosanitario implementadas.

Index Terms—Programación Orientada a Objetos, Multithreading, Gemelo Digital, Agricultura 4.0, Invernadero Inteligente.



Figura 1.

I. INTRODUCCIÓN

El auge de la agricultura protegida en el contexto de la Agricultura 4.0 ha cambiado drásticamente las reglas del juego en la producción mundial de alimentos [1]. Hoy en día, un invernadero ya no es una simple estructura pasiva de plástico o vidrio; es un ecosistema dinámico, conectado y altamente tecnológico [2]. Monitorear y simular este tipo de entornos requiere de sistemas informáticos robustos, capaces de reaccionar en tiempo real frente a múltiples variables concurrentes y fenómenos físicos entrelazados. Este proyecto nace con el propósito de diseñar una aplicación de escritorio avanzada, intuitiva y modular que actúe como un gemelo digital del clima interno del invernadero [3].

A diferencia de los enfoques de programación tradicionales y lineales, este software ha sido construido integralmente bajo el paradigma de la Programación Orientada a Objetos (POO) [4]. Esto permite modelar cada fenómeno físico, cada sensor y cada cultivo como una entidad autónoma con un estado interno protegido y un comportamiento lógico bien definido. El resultado de esta ingeniería es una plataforma multicapa que simula variables ambientales clave (temperatura, humedad relativa y luminosidad), integra datos en vivo mediante APIs meteorológicas externas, y predice con base matemática el riesgo de proliferación de enfermedades y plagas en cultivos de alta demanda agronómica como el tomate, la lechuga y el pimiento.

II. OBJETIVOS DEL PROYECTO

II-A. Objetivo General

Desarrollar un sistema computacional integrado de simulación y monitoreo microclimático para un invernadero inteli-

gente mediante el uso del lenguaje de programación Python, estructurado bajo una arquitectura modular orientada a objetos, hilos de ejecución concurrentes y consumo asíncrono de APIs externas, con el fin de predecir riesgos biológicos y asegurar el equilibrio termodinámico de múltiples cultivos simultáneos.

II-B. Objetivos Específicos

- Diseñar una arquitectura de datos centralizada e independiente mediante la implementación del patrón de diseño Singleton, logrando aislar eficientemente la lógica física (backend) de la interfaz gráfica de usuario (frontend).
- Implementar programación concurrente (*Multithreading*) para procesar el ciclo de simulación temporal interna en paralelo con las consultas de red asíncronas a la API de Open-Meteo.
- Construir un entorno visual adaptativo e intuitivo empleando elementos gráficos interactivos como lienzos dinámicos en Matplotlib y medidores de aguja vectoriales basados en geometría analítica.
- Modelar un algoritmo probabilístico predictivo que evalúe y alerte en tiempo real sobre el riesgo biológico de plagas e infecciones fúngicas específicas por cada cultivo basados en el estrés higrométrico y térmico.

III. MARCO TEÓRICO Y CONCEPTUAL

Para soportar el diseño arquitectónico de una aplicación de esta envergadura y asegurar su escalabilidad, se aplicaron de forma rigurosa pilares del desarrollo de software moderno:

Tabla I
SECUENCIA DE EJECUCIÓN Y FLUJO DE DATOS DEL SISTEMA CONCURRENTE MULTICAPA

Paso	Componente / Evento	Acción y Flujo de Datos Relacionado
1	Inicio de Aplicación	main_gui.py invoca a get_manager() para capturar el Singleton de datos.
2	Inicialización de Hilos	SensorManager levanta dos subprocesos independientes de fondo (<i>Daemon Threads</i>).
3	Hilo Meteorológico Exterior	_update_exterior_loop realiza peticiones HTTP REST a la API Open-Meteo.
4	Hilo de Simulación Interna	_simulate_step() calcula cada segundo las inercias de Temperatura, Humedad y Luz.
5	Disparo de Callbacks	Al calcular los nuevos valores, el backend notifica a todas las interfaces registradas.
6	Actualización de Interfaz	main_gui.py captura los nuevos datos actualizando los medidores y redibujando gráficos.
7	Cálculo de Riesgos	Las sub-GUIs de Cultivos consultan al mánager para procesar las probabilidades de plagas.
8	Análisis de Intersección	general_gui.py evalúa matemáticamente si las tres especies pueden cohabitar en paz.
9	Cierre del Sistema	Se detienen de forma segura los hilos asíncronos previniendo bloqueos.

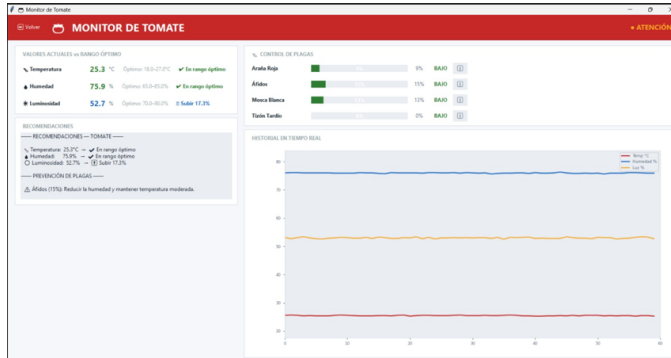


Figura 3.

V. CONCLUSIONES

La utilización del paradigma de Programación Orientada a Objetos resultó indispensable para evitar la dispersión de datos. El aislamiento completo entre el motor físico de simulación y la interfaz visual garantiza un mantenimiento limpio del software y escalabilidad absoluta.

El uso de *Multithreading* (múltiples hilos concurrentes) resolvió con éxito las latencias de red inherentes. Separar la comunicación con servidores externos de la renderización visual asegura una experiencia de usuario sumamente fluida, robusta y libre de bloqueos críticos.

El modelado matemático y predictivo de perfiles botánicos y riesgos fitosanitarios transforma esta aplicación de un simple tablero gráfico a una herramienta analítica preventiva, demostrando el enorme potencial que ofrece el desarrollo de software estructurado aplicado al agro colombiano.

REFERENCIAS

- [1] A. Walter, R. Finger, R. Huber, y W. Kurth, "Smart farming is key to developing sustainable agriculture," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 114, no. 24, pp. 6148–6150, 2017.
- [2] L. Shamshiri *et al.*, "Advances in greenhouse automation and controlled environment agriculture: A transition to plant factories and urban agriculture," *Int. J. Agric. Biol. Eng.*, vol. 11, no. 1, pp. 1–22, 2018.
- [3] C. Pyliandis, S. Osinga, y I. N. Athanasiadis, "Introducing digital twins to agriculture," *Computers and Electronics in Agriculture*, vol. 184, p. 105942, 2021.
- [4] M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA, USA: O'Reilly Media, 2013.
- [5] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

- [6] A. Shalloway y J. R. Trott, *Design Patterns Explained: A New Perspective on Object-Oriented Design*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [7] D. Beazley, "Understanding the Python GIL," in *PyCON Python Conference*, Atlanta, GA, 2010.
- [8] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [9] P. E. B. S. C. C. M. *et al.*, "Predictive models for plant diseases and pests in greenhouse environments," *Journal of Agricultural Science*, vol. 155, no. 3, pp. 430–445, 2017.
- [10] F. P. O'Mara, "The role of grasslands in food security and climate change," *Annals of Botany*, vol. 110, no. 6, pp. 1263–1270, 2012.